Constructor Types - Default, Parameterized, Copy, and Move

Theory, Edge Cases & Code Examples

- ✓ Object Initialization Mechanisms
- ✓ Default Values and Initialization Methods
- ✓ Initialization Order and Pitfalls
- ✓ Implicit Default Constructor Suppression
- ✓ Initialization Order vs Initializer List Order

→ *Swipe to learn* →

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 1. Constructor Types - Object Initialization Mechanisms (1/2)

Constructor overview:

Constructor Type	Signature	When Called	Purpose	Auto-Generated
Default	`T()` or `T(args=defaults)`	No arguments provided	Initialize with default values	✅ Only if no other constructors exist
Parameterized	`T(Type1 arg1, Type2 arg2, ...)`	With specific arguments	Initialize with custom values	❌ Never auto-generated
Copy	`T(const T& other)`	From lvalue of same type	Create duplicate of existing object	✅ Unless move/copy/destructor user-defined
Move	`T(T&& other)` noexcept	From rvalue of same type	Transfer resources from temporary	✅ Only if no special members user-defined

Compiler-generated constructor rules (Rule of Five):

You Declare	Default	Copy	Move	Reason
Nothing	✅ Generated	✅ Generated	✅ Generated	Full automatic support
Any constructor	❌ Suppressed	✅ Generated	✅ Generated	Explicit construction required
Copy constructor	❌ Suppressed	User-defined	❌ Suppressed	Resource management indicated
Move constructor	❌ Suppressed	❌ Suppressed	User-defined	Resource management indicated
Destructor	❌ Suppressed	✅ Generated (deprecated)	❌ Suppressed	Resource management indicated
Copy assignment	❌ Suppressed	✅ Generated	❌ Suppressed	Resource management indicated

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 1. Constructor Types - Object Initialization Mechanisms (2/2)

Key insight: Declaring any constructor suppresses automatic default constructor generation. Use `= default` to restore: `T() = default;`

Code example:

```
class Resource {
public:
    Resource() { /* default */ }                    // Default constructor
    Resource(int size) { /* param */ }              // Parameterized
    Resource(const Resource& other) { /* copy */ }  // Copy constructor
    Resource(Resource&& other) noexcept { /* move */ }  // Move constructor
    ~Resource() { /* cleanup */ }                   // Destructor
};
```

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 2. Member Initialization - Default Values and Initialization Methods (1/3)

Default values by type and context:

Member Type	Non-Static Local/Member Object	Static/Global Object	Safe?
Built-in types (int, float, char)	❌ **Garbage (undefined behavior)**	✅ Zero-initialized	❌ Must explicitly initialize
Pointers (T*)	❌ **Garbage (dangerous)**	✅ nullptr	❌ Must explicitly initialize
Class types (std::string, std::vector)	✅ Default constructor called	✅ Default constructor called	✅ Automatic initialization
const members	Must initialize (initializer list)	Must initialize	N/A - Requires explicit initialization
Reference members	Must initialize (initializer list)	Must initialize	N/A - Requires explicit initialization

CRITICAL: Built-in types in non-static objects contain **garbage values** unless explicitly initialized. This is a common source of undefined behavior, security vulnerabilities, and heisenbugs.

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 2. Member Initialization - Default Values and Initialization Methods (2/3)

Member initialization methods comparison:

Method	Syntax	Performance	When to Use	Restrictions
Constructor body assignment	`T() { member = value; }`	❌ Slow (default-construct + assign)	Avoid if possible	Cannot use for const/ref members
Member initializer list	`T() : member(value) {}`	✅ Optimal (direct initialization)	**Preferred method**	Required for const/ref/no-default-ctor
In-class initializer (C++11+)	`int member = 0;`	✅ Optimal	Default values for all constructors	Overridden by initializer list
Delegating constructor (C++11+)	`T() : T(default_value) {}`	✅ Optimal	Reduce code duplication	Cannot mix with member initialization

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 2. Member Initialization - Default Values and Initialization Methods (3/3)

Code example - initialization methods:

```
class Configuration {
    // In-class initializers (C++11+) - provide defaults
    int timeout = 30;
    bool enabled = true;
    std::string mode = "standard";
    const int maxConnections;  // Must initialize in initializer list

public:
    // ✅ Good: Member initializer list (direct initialization)
    Configuration(int max) : maxConnections(max) {
        // timeout, enabled, mode use in-class defaults
    }

    // ✅ Good: Overrides defaults with initializer list
    Configuration(int max, int t, bool e)
        : maxConnections(max), timeout(t), enabled(e) {}

    // ❌ Bad: Assignment in constructor body (inefficient)
    // Configuration(int max) {
    //     maxConnections = max;  // ❌ Error: cannot assign to const
    //     mode = "custom";       // ❌ Inefficient: default-construct then assign
    // }
};
```

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 3. Initialization Order and Pitfalls (1/2)

Member initialization order (CRITICAL):

Order Step	What Initializes	Determined By	Common Mistake
1. Virtual base classes	Most derived class controls initialization	Inheritance hierarchy	Forgetting virtual base init
2. Direct base classes	Left to right in declaration order	Class declaration	Wrong base init order assumed
3. Member variables	**Declaration order (NOT initializer list order)**	Class declaration	**Relying on list order instead**
4. Constructor body	Executes after all members initialized	Code execution	Assuming members uninitialized in body

DANGER: Members are initialized in **declaration order**, not initializer list order. Relying on list order causes undefined behavior.

Constructor Types - Default, Parameterized, Copy, and Move

Theory

📌 3. Initialization Order and Pitfalls (2/2)

Code example - initialization order bug:

```
class Dangerous {
    int second;  // Declared first
    int first;   // Declared second

public:
    // ❌ Bug: initializer list order doesn't matter
    Dangerous(int x) : first(x), second(first + 1) {
        // Actual order: second initialized first (declaration order)
        // second = first + 1, but first is UNINITIALIZED
        // Result: second contains garbage
    }

    // ✅ Fix: Match initializer list to declaration order
    Dangerous(int x) : second(x), first(second + 1) {
        // Now safe: second initialized first with x,
        // then first initialized with second + 1
    }
};
```

Best practices for avoiding initialization bugs:

Practice	Why	Example
Match initializer list to declaration order	Prevents using uninitialized members	List members top-to-bottom as declared
Avoid inter-member dependencies	Safer initialization	Each member initialized independently
Always initialize built-in types	Prevent garbage values	`int x = 0;` or `: x(0)`
Use const for values that shouldn't change	Compile-time enforcement	`const int maxSize = 100;`
Use initializer lists, not body assignment	Required for const/ref, more efficient	`: member(value)`

Key takeaway: Always use member initializer lists for efficiency and correctness. Members initialize in **declaration order**, not list order. Explicitly initialize all built-in types to avoid undefined behavior.

Constructor Types - Default, Parameterized, Copy, and Move

Edge Cases

📌 1. Implicit Default Constructor Suppression

When you define any constructor (even a parameterized one), the compiler no longer generates a default constructor. This is a frequent source of compilation errors when code expects default construction but only parameterized constructors exist.

```
class Resource {
public:
    Resource(std::string name) {  // ✅ Parameterized constructor
        std::cout << "Resource: " << name << "\n";
    }
};

int main() {
    // Resource r;  // ❌ Error: no default constructor
    Resource r("data");  // ✅ OK
}
```

This behavior forces explicit construction and prevents accidental uninitialized objects. To restore default construction capability, explicitly define a default constructor using `= default` or provide your own implementation. This design choice reflects C++'s philosophy of no implicit overhead—if you need special construction, you must be explicit.

Constructor Types - Default, Parameterized, Copy, and Move

Edge Cases

📌 2. Initialization Order vs Initializer List Order

A subtle and dangerous pitfall: members are initialized in the order they're **declared in the class**, not the order specified in the initializer list. The compiler will warn about this in some cases, but not all.

```
class Tricky {
    int first;
    int second;
public:
    Tricky(int x) : second(x), first(second) {  // ❌ Dangerous!
        // first is initialized before second
        // first gets garbage value from uninitialized second
    }
};
```

This can lead to using uninitialized values, accessing null pointers, or other undefined behavior. The fix is simple: match initializer list order to declaration order, or better yet, avoid dependencies between member initializations. Some compilers warn with `-Wreorder`, but it's not guaranteed. Always initialize members in declaration order to avoid confusion and bugs.

Constructor Types - Default, Parameterized, Copy, and Move

Edge Cases

📌 3. Const and Reference Members Require Initializer Lists

Const and reference members cannot be assigned—they must be initialized at construction. This mandates using member initializer lists; assignment in the constructor body is illegal and won't compile.

```
class Validator {
    const int maxValue;  // Cannot be reassigned
    int& externalCounter;  // Must bind to existing int

public:
    Validator(int max, int& counter)
        : maxValue(max), externalCounter(counter) {  // ✅ Must use initializer list
        // maxValue = max;  // ❌ Error: cannot assign to const
    }
};
```

This restriction reflects C++'s type system semantics—references are aliases that must be bound at creation, and const objects cannot be modified after initialization. Attempting to use assignment in the constructor body fails because the members must be initialized before the body executes, and const/reference semantics prohibit subsequent modification.

Constructor Types - Default, Parameterized, Copy, and Move

Edge Cases

📌 4. Copy Elision and Return Value Optimization

Modern compilers perform copy elision and return value optimization (RVO), eliminating copy/move constructors even when they would normally be called. In C++17, copy elision is mandatory in certain cases, changing program behavior.

```
class Logger {
public:
    Logger() { std::cout << "Construct\n"; }
    Logger(const Logger&) { std::cout << "Copy\n"; }
    Logger(Logger&&) { std::cout << "Move\n"; }
};

Logger create() {
    return Logger();  // ❌ C++17: no copy/move, mandatory elision
}

int main() {
    Logger log = create();  // Output: just "Construct"
}
```

Pre-C++17, you might see "Construct" then "Copy" or "Move". C++17 guarantees elision when returning a temporary, so only the constructor runs. This optimization changes observable behavior but improves performance dramatically. Understanding when elision happens helps explain surprising output in constructor counting exercises and affects how you design classes.

Constructor Types - Default, Parameterized, Copy, and Move

Edge Cases

📌 5. Delegating Constructor Cycle Detection

C++11 allows constructors to delegate to other constructors, reducing code duplication. However, creating circular delegation causes compilation errors or infinite recursion.

```
class Bad {
public:
    Bad() : Bad(0) {}  // ❌ Calls Bad(int)
    Bad(int x) : Bad() {}  // ❌ Calls Bad() - circular!
};
```

The compiler usually detects direct cycles, but indirect cycles through multiple constructors might not be caught, leading to stack overflow at runtime. Delegating constructors must form a directed acyclic graph with exactly one "final" constructor that performs actual initialization. This constructor should not delegate to any other. Use delegation to avoid repeating initialization logic, but design the delegation hierarchy carefully.

Constructor Types - Default, Parameterized, Copy, and Move

Edge Cases

📌 6. Move Constructor Automatic Generation Rules

The compiler automatically generates a move constructor only under very specific conditions: when no user-declared copy constructor, copy assignment operator, move assignment operator, or destructor exists. Defining any of these suppresses automatic move generation.

```
class HasDestructor {
public:
    ~HasDestructor() {}  // ❌ User-declared destructor
    // Move constructor NOT automatically generated
};

class HasCopy {
public:
    HasCopy(const HasCopy&) {}  // ❌ User-declared copy constructor
    // Move constructor NOT automatically generated
};
```

This is part of the Rule of Five—if you declare any of the five special member functions (destructor, copy constructor, copy assignment, move constructor, move assignment), you should probably explicitly define or delete all five. Relying on automatic generation after declaring one can lead to inefficient copying when moves were expected, or worse, incorrect resource management.

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 1. Default Constructor Behavior

This demonstrates the danger of not initializing built-in types. The value contains whatever bits happened to be in that memory location. The fix is to use a member initializer list: `Counter() : value(0) {}` or in-class initialization: `int value = 0;`. This undefined behavior is a common source of heisenbugs that appear and disappear based on memory state.

```
class Counter {
    int value;  // ❌ Uninitialized for non-static objects
public:
    Counter() {
        std::cout << "Default constructor, value = " << value << "\n";
    }
};

int main() {
    Counter c;  // value contains garbage
}
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 2. Parameterized Constructor with Validation

This shows how to perform validation during initialization. The ternary operator in the initializer list allows conditional initialization. For more complex validation, consider a factory function or throwing an exception if invalid parameters should prevent object creation entirely.

```
class BankAccount {
    std::string owner;
    double balance;

public:
    BankAccount(std::string name, double initial)
        : owner(name), balance(initial < 0 ? 0 : initial) {
        // ✅ Member initializer list with validation
        std::cout << "Account for " << owner
                  << " with balance " << balance << "\n";
    }
};

int main() {
    BankAccount acc("Alice", 1000.0);
    BankAccount invalid("Bob", -50.0);  // balance becomes 0
}
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 3. Member Initializer List vs Constructor Body

The first version default-constructs `data` (empty string), then assigns the value. The second directly constructs `data` with the value, avoiding the temporary empty string. For simple types like int, the difference is negligible, but for complex types like std::string or std::vector, the initializer list can significantly improve performance by avoiding unnecessary default construction and allocation.

```
class String {
    std::string data;
public:
    // ❌ Inefficient: default-constructs data, then assigns
    String(const char* str) {
        data = str;
    }

    // ✅ Efficient: directly constructs data with str
    String(const char* str) : data(str) {}
};
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 4. In-Class Member Initializers

In-class initializers (C++11+) provide default values that are used when not overridden by a constructor's initializer list. This reduces duplication when you have multiple constructors that share common default values. The initializer list takes precedence, allowing selective override of specific members while using defaults for others.

```
class Configuration {
    int timeout = 30;  // ✅ Default value
    bool enabled = true;
    std::string mode = "standard";

public:
    Configuration() {}  // Uses default values

    Configuration(int t) : timeout(t) {
        // ✅ Overrides timeout, uses defaults for others
    }

    Configuration(int t, bool e, std::string m)
        : timeout(t), enabled(e), mode(m) {
        // ✅ Overrides all values
    }
};
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 5. Copy Constructor Deep vs Shallow Copy

This is the classic example of why custom copy constructors are needed for classes managing resources. The shallow copy creates two objects pointing to the same memory, causing double-delete when both are destroyed. The deep copy allocates new memory and copies the data, giving each object independent ownership. This is a fundamental pattern in resource management.

```
class DynamicArray {
    int* data;
    size_t size;

public:
    DynamicArray(size_t n) : size(n), data(new int[n]) {}

    // ❌ Shallow copy (compiler-generated would do this)
    DynamicArray(const DynamicArray& other)
        : size(other.size), data(other.data) {
        // Both objects point to same array - double delete!
    }

    // ✅ Deep copy
    DynamicArray(const DynamicArray& other)
        : size(other.size), data(new int[other.size]) {
        std::copy(other.data, other.data + size, data);
    }

    ~DynamicArray() { delete[] data; }
};
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 6. Move Constructor Resource Transfer

The move constructor transfers ownership of resources from the source object (which is being destroyed) to the new object. The source is left in a valid but unspecified state—typically nulled out. This avoids expensive copying of resources like file handles, sockets, or large buffers. Mark move constructors `noexcept` when possible for optimal container performance.

```
class FileHandle {
    FILE* file;
    std::string filename;

public:
    FileHandle(std::string name) : filename(name) {
        file = fopen(name.c_str(), "r");
    }

    // ✅ Move constructor - steals resources
    FileHandle(FileHandle&& other) noexcept
        : file(other.file), filename(std::move(other.filename)) {
        other.file = nullptr;  // ✅ Leave other in valid state
    }

    ~FileHandle() {
        if (file) fclose(file);
    }
};

FileHandle create() {
    return FileHandle("data.txt");  // ✅ Move, not copy
}
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 7. Delegating Constructors

Delegating constructors (C++11+) reduce code duplication by having one constructor call another. The delegating constructor does nothing else—initialization logic is in the target constructor. This ensures consistent initialization and validation across different construction paths. The target constructor executes completely before the delegating constructor's body (if any) runs.

```
class Rectangle {
    int width, height;

    void validate() {
        if (width < 0) width = 0;
        if (height < 0) height = 0;
    }

public:
    // Main constructor
    Rectangle(int w, int h) : width(w), height(h) {
        validate();
        std::cout << "Rectangle: " << width << "x" << height << "\n";
    }

    // ✅ Delegate to main constructor
    Rectangle() : Rectangle(0, 0) {}

    // ✅ Square constructor delegates
    Rectangle(int side) : Rectangle(side, side) {}
};
```

Constructor Types - Default, Parameterized, Copy, and Move

Code Examples

📌 8. Explicit Constructor to Prevent Implicit Conversion

The `explicit` keyword prevents implicit conversions from the constructor's parameter type to the class type. Without it, `Array a = 10;` would implicitly call `Array(10)`, which might be unexpected and lead to bugs. Use `explicit` for single-parameter constructors unless implicit conversion is genuinely desired (e.g., std::string from const char*).

```
class Array {
    int* data;
    size_t size;

public:
    explicit Array(size_t n) : size(n), data(new int[n]) {}

    ~Array() { delete[] data; }
};

void process(Array arr) {
    // ...
}

int main() {
    // Array a = 10;  // ❌ Error: explicit prevents implicit conversion
    Array a(10);      // ✅ OK: explicit construction

    // process(20);   // ❌ Error: no implicit conversion
    process(Array(20));  // ✅ OK: explicit construction
}
```

What You Learned

Key Takeaways

- ✓ Object Initialization Mechanisms
- ✓ Default Values and Initialization Methods
- ✓ Initialization Order and Pitfalls
- ✓ Implicit Default Constructor Suppression

→ Swipe to learn →